Textbooks Are All You Need

Microsoft Research

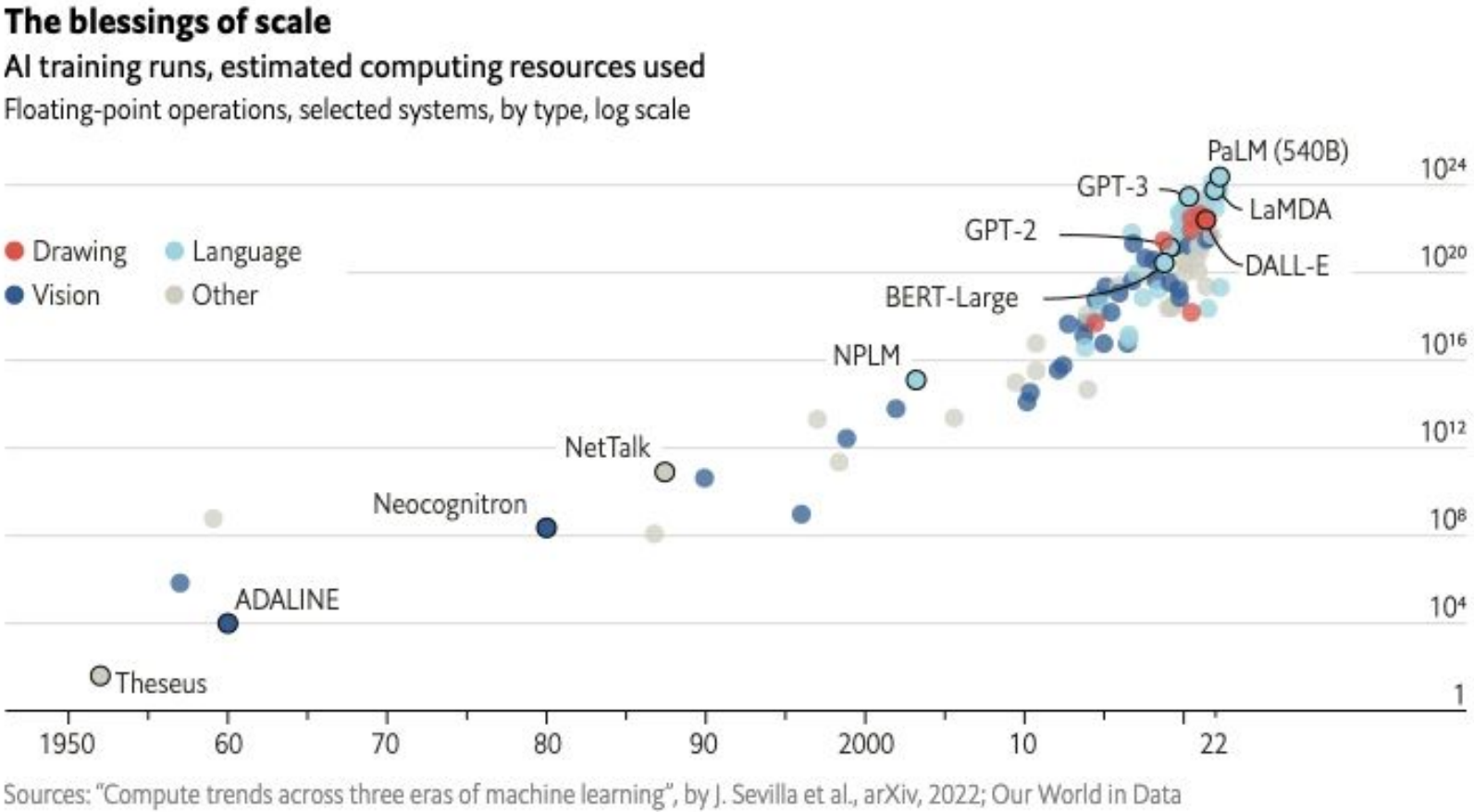
자연어처리팀

황예진(발표자), 황지현, 정강민, 변현정, 조해창

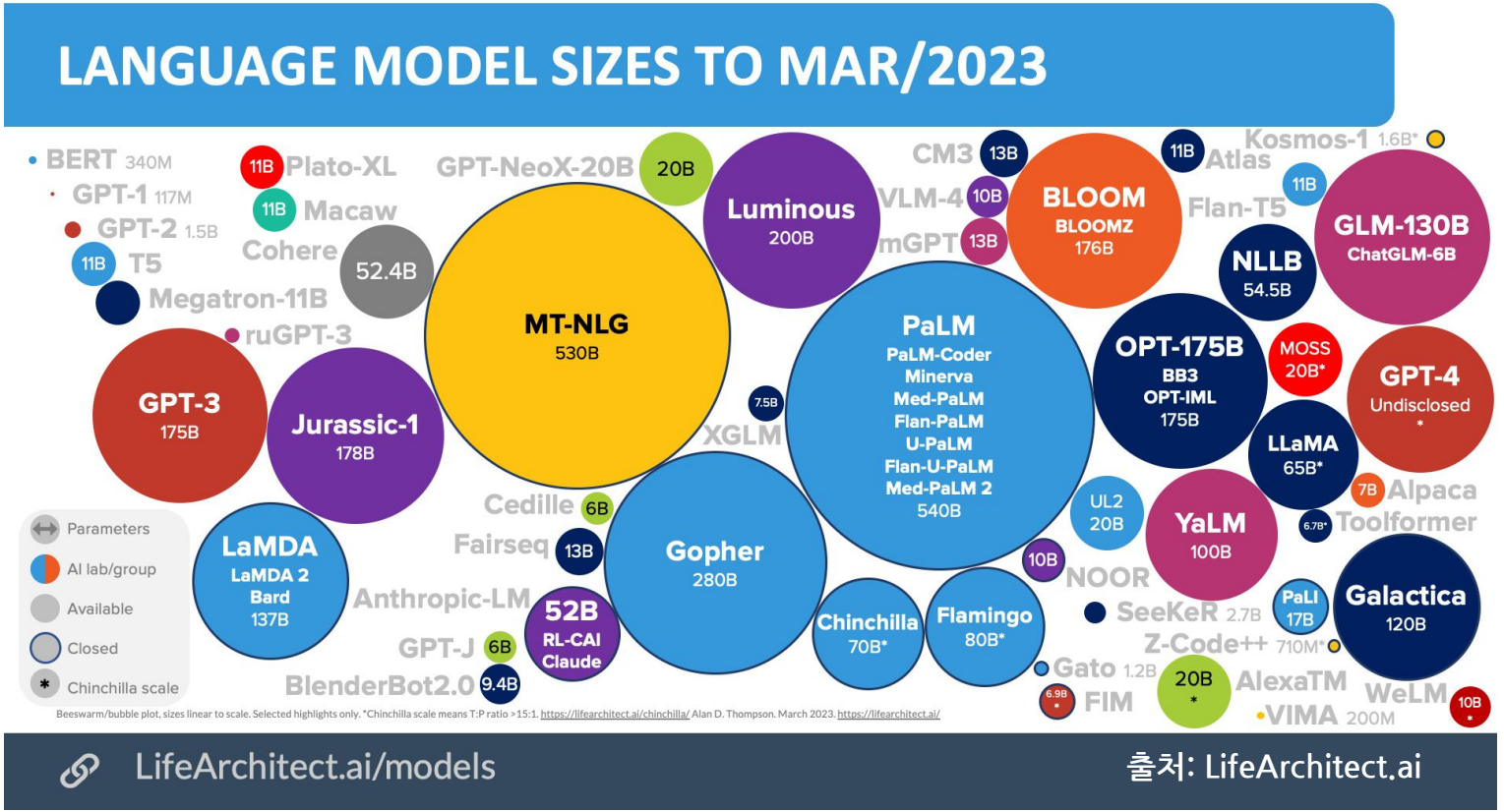
Content

overview

Overview


The blessings of scale
AI training runs, estimated computing resources used
Floating-point operations, selected systems, by type, log scale
Drawing
Language
Vision
Other
PaLM (540B)
GPT-3
LaMDA
GPT-2
DALL-E
BERT-Large
NPLM
NetTalk
Neocognitron
ADALINE
Theseus
10^{24}
10^{20}
10^{16}
10^{12}
10^{8}
10^{4}
1
1950
60
70
80
90
2000
10
22
Sources: "Compute trends across three eras of machine learning", by J. Sevilla et al., arXiv, 2022; Our World in Data

Overview


LANGUAGE MODEL SIZES TO MAR/2023
BERT 340M
GPT-1 117M
GPT-2 1.5B
11B T5
Megatron-11B
11B Plato-XL
11B Macaw
Cohere
52.4B
ruGPT-3
GPT-3 175B
Jurassic-1 178B
MT-NLG 530B
GPT-NeoX-20B 20B
Luminous 200B
CM3 13B
VLM-4 10B
mGPT 13B
BLOOM BLOOMZ 176B
11B Atlas
Kosmos-1 1.6B*
11B Flan-T5
GLM-130B ChatGLM-6B
NLLB 54.5B
PaLM PaLM-Coder Minerva Med-PaLM Flan-PaLM U-PaLM Flan-U-PaLM Med-PaLM 2 540B
7.5B XGLM
OPT-175B BB3 OPT-IML 175B
MOSS 20B*
GPT-4 Undisclosed *
LLaMA 65B*
7B Alpaca
6.7B* Toolformer
Cedille 6B
Fairseq 13B
LaMDA LaMDA 2 Bard 137B
Anthropic-LM
52B RL-CAI Claude
GPT-J 6B
BlenderBot2.0 9.4B
Gopher 280B
Chinchilla 70B*
Flamingo 80B*
UL2 20B
YaLM 100B
10B NOOR
SeeKeR 2.7B
PaLI 17B
Galactica 120B
Z-Code++ 710M*
Gato 1.2B
6.9B* FIM
20B *
AlexaTM
WeLM 10B*
VIMA 200M
Parameters
AI lab/group
Available
Closed
* Chinchilla scale
Beeswarm/bubble plot, sizes linear to scale. Selected highlights only. *Chinchilla scale means T:P ratio >15:1. https://lifearchitect.ai/chinchilla/ Alan D. Thompson. March 2023. https://lifearchitect.ai/
LifeArchitect.ai/models
출처: LifeArchitect.ai

Overview

Textbooks

clear, self-contained,
instructive and balanced

Overview

Model	Size	Training tokens	Score	HumanEval
CodeGen-Mono-350M [NPH+23]	350M	577B	0.19	12.8%
CodeGen-Mono-16.1B [NPH+23]	16.1B	577B	0.38	29.3%
Replit [Rep23]	2.7B	525B	0.37	21.9%
StarCoder [LAZ+23]	15.5B	1T	0.51	33.6%
phi-1-base	1.3B	7B	0.37	29%
phi-1-small	350M	7B	0.45	45%
phi-1	1.3B	7B	0.52	50.6%

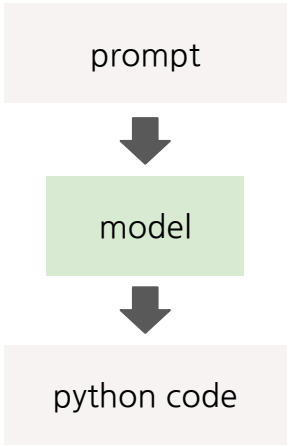
Table 2: LLM graded Understanding scores on 50 new unconventional coding problems.

10x smaller in model size and 100x smaller in dataset size

Overview

Task

Python code generate



Prompt:

```
""" Write a PyGame.  There is a ball.  At every iteration, (1).  The x-axis of the ball
increases by a random integer between (-10, 10), (2).  The y-axis of the ball increases
by a random integer between (-20, 20).  The x-axis of the ball should stay inside 0-400,
and y-axis of the ball should stay inside 0-600.  When the user press 'space', set the
x-axis of the ball to 200 and y-axis of the ball to 400.
"""
```

phi-1

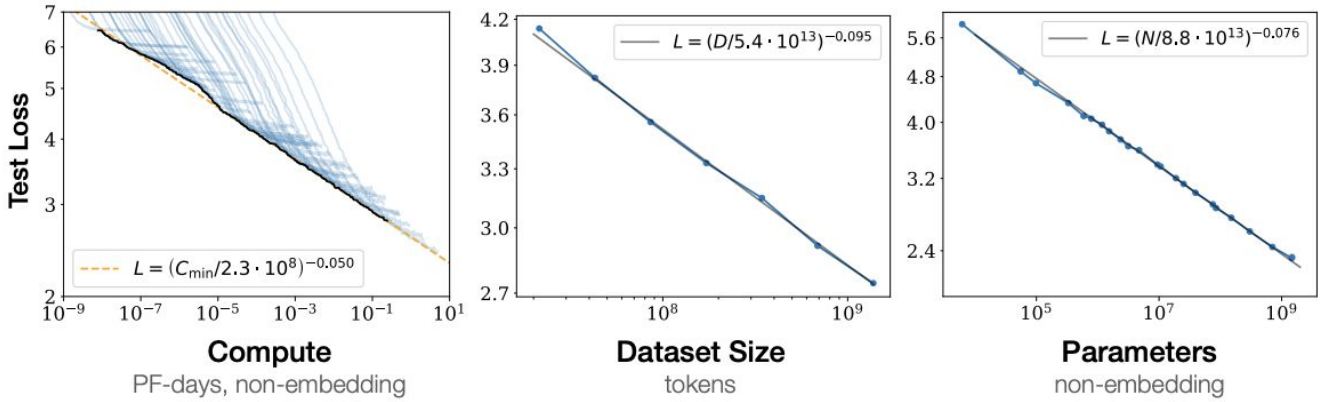
```
...
screen = pygame.display.set_mode
    ((400, 600))
x, y = 0, 0
while True:
    for event in pygame.event.get
    ():
        ...
        elif event.type == pygame
    .KEYDOWN:
            if event.key ==
    pygame.K_SPACE:
                x = 200
                y = 400
    x += random.randint(-10, 10)
    y += random.randint(-20, 20)
    if x < 0:
        x = 0
    elif x > 400:
        x = 400
    if y < 0:
        y = 0
    elif y > 600:
        y = 600
    screen.fill((255, 255, 255))
    pygame.draw.circle(screen,
    (255, 0, 0), (x, y), 10)
    pygame.display.update()
```

Introduction

1. Introduction

Scaling laws

performance improves as one scales up either the amount of **compute** or the **size of the network**



1. Introduction

(not quantity) the ***quality*** of data

Date	Model	Model size	Dataset size (Tokens)	HumanEval (Pass@1)	MBPP (Pass@1)
2021 Jul	Codex-300M [CTJ+21]	300M	100B	13.2%	-
2021 Jul	Codex-12B [CTJ+21]	12B	100B	28.8%	-
2022 Mar	CodeGen-Mono-350M [NPH+23]	350M	577B	12.8%	-
2022 Mar	CodeGen-Mono-16.1B [NPH+23]	16.1B	577B	29.3%	35.3%
2022 Apr	PaLM-Coder [CND+22]	540B	780B	35.9%	47.0%
2022 Sep	CodeGeeX [ZXZ+23]	13B	850B	22.9%	24.4%
2022 Nov	GPT-3.5 [Ope23]	175B	N.A.	47%	-
2022 Dec	SantaCoder [ALK+23]	1.1B	236B	14.0%	35.0%
2023 Mar	GPT-4 [Ope23]	N.A.	N.A.	67%	-
2023 Apr	Replit [Rep23]	2.7B	525B	21.9%	-
2023 Apr	Replit-Finetuned [Rep23]	2.7B	525B	30.5%	-
2023 May	CodeGen2-1B [NHX+23]	1B	N.A.	10.3%	-
2023 May	CodeGen2-7B [NHX+23]	7B	N.A.	19.1%	-
2023 May	StarCoder [LAZ+23]	15.5B	1T	33.6%	52.7%
2023 May	StarCoder-Prompted [LAZ+23]	15.5B	1T	40.8%	49.5%
2023 May	PaLM 2-S [ADF+23]	N.A.	N.A.	37.6%	50.0%
2023 May	CodeT5+ [WLG+23]	2B	52B	24.2%	-
2023 May	CodeT5+ [WLG+23]	16B	52B	30.9%	-
2023 May	InstructCodeT5+ [WLG+23]	16B	52B	35.0%	-
2023 Jun	WizardCoder [LXZ+23]	16B	1T	57.3%	51.8%
2023 Jun	**phi-1**	1.3B	7B	50.6%	55.5%

1. Introduction

HumanEval

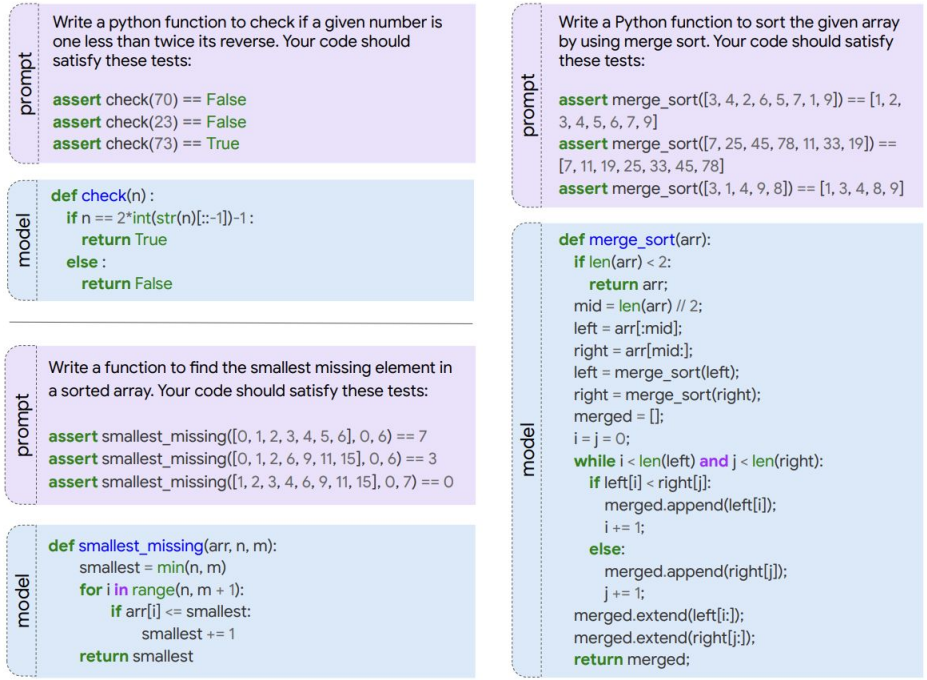
- **164** handwritten programming problems
- each problem includes a function signature, docstring, body, and **several unit tests**

{"task_id": "HumanEval/0", "prompt": "from typing import List₩n₩n₩ndef has_close_elements(numbers: List[float], threshold: float) -> bool:₩n ₩"₩"₩" Check if in given list of numbers, are any two numbers closer to each other than₩n given threshold.₩n >>> has_close_elements([1.0, 2.0, 3.0], 0.5)₩n False₩n >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)₩n True₩n ₩"₩"₩"₩n", "entry_point": "has_close_elements", "canonical_solution": " for idx, elem in enumerate(numbers):₩n for idx2, elem2 in enumerate(numbers):₩n if idx != idx2:₩n distance = abs(elem - elem2)₩n if distance < threshold:₩n return True₩n₩n return False₩n", "test": "₩n₩nMETADATA = {₩n 'author': 'jt',₩n 'dataset': 'test'₩n}₩n₩n₩ndef check(candidate):₩n assert candidate([1.0, 2.0, 3.9, 4.0, 5.0, 2.2], 0.3) == True₩n assert candidate([1.0, 2.0, 3.9, 4.0, 5.0, 2.2], 0.05) == False₩n assert candidate([1.0, 2.0, 5.9, 4.0, 5.0], 0.95) == True₩n assert candidate([1.0, 2.0, 5.9, 4.0, 5.0], 0.8) == False₩n assert candidate([1.0, 2.0, 3.0, 4.0, 5.0, 2.0], 0.1) == True₩n assert candidate([1.1, 2.2, 3.1, 4.1, 5.1], 1.0) == True₩n assert candidate([1.1, 2.2, 3.1, 4.1, 5.1], 0.5) == False₩n₩n"}

1. Introduction

MBPP (Mostly Basic Programming Problems)

- **974** short Python programs constructed by crowd-sourcing
- a single self-contained Python function, **three test cases**



Training details and the importance of high-quality data

2. Training details and the importance of high-quality data

기존 코드 데이터셋의 문제점

- not self-contained
- do not involve any meaningful computation
- do contain algorithmic logic are often buried inside complex or poorly documented functions
- skewed towards certain topics or use cases

2. Training details and the importance of high-quality data

기존 코드 데이터셋의 문제점

- not self-contained
- do not involve any meaningful cor
- do contain algorithmic logic are of
- skewed towards certain topics or u

▼ depend on other file('data.csv')

```
import data_processor
import pandas as pd
import matplotlib.pyplot as plt

# Read data from a CSV file
data = pd.read_csv('data.csv')

# Process data using a custom module
processed_data = data_processor.process_data(data)

# Perform analysis and create a plot
plt.plot(processed_data)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Data Analysis')
plt.show()
```

tions

2. Training details and the importance of high-quality data

기존 코드 데이터셋의 문제점

- not self-contained
- do not involve any meaningful computation
- do contain algorithmic logic are often buried
- skewed towards certain topics or use cases

▼ gui configuration

```python
# gui_configurator.py

import gui_elements
import json

# Load GUI configuration
with open('gui_config.json', 'r') as config_file:
    gui_config = json.load(config_file)

# Create and configure GUI elements based on configuration
for element_data in gui_config['elements']:
    element = gui_elements.create_element(element_data)
    element.configure(element_data['options'])
    element.display()
```

2. Training details and the importance of high-quality data

기존 코드 데이터셋의 문제점

- not self-contained
- do not involve any meaningful computation
- do contain algorithmic logic are often buried inside complex or poorly documented functions
-

```python
def process_data(data):
    # Perform some initial preprocessing
    cleaned_data = clean_data(data)

    # Apply a complex algorithmic logic
    result = []
    for item in cleaned_data:
        if check_condition(item):
            intermediate_result = []
            for value in item.values():
                if complex_transform(value):
                    intermediate_result.append(value)
            result.append(intermediate_result)

    # Apply further post-processing
    final_result = post_process(result)

    return final_result
```

```python
def clean_data(data):
    # Perform data cleaning operations
    # ...

def check_condition(item):
    # Check if item meets a certain condition
    # ...

def complex_transform(value):
    # Apply a complex transformation to the value
    # ...

def post_process(result):
    # Perform final post-processing on the result
    # ...

# Usage
input_data = [...]  # Some input data
output = process_data(input_data)
print(output)
```

2. Training details and the importance of high-quality data

기존 코드 데이터셋의 문제점

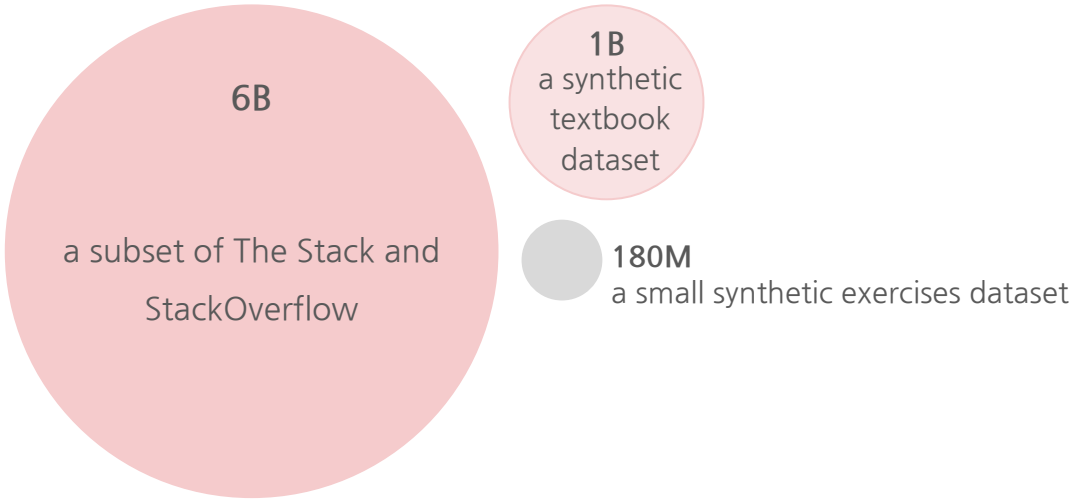
- not self-contained
- do not involve any meaningful computation
- do contain algorithmic logic are often buried inside complex or poorly documented functions
- skewed towards certain topics or use cases

→ a lot of noise, ambiguity, and incompleteness in the data

2. Training details and the importance of high-quality data

양질의 교과서 같은 학습 데이터셋을 세 가지 형태로 준비

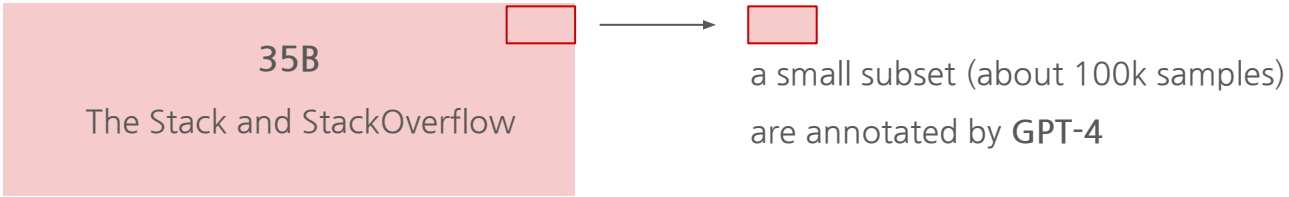
- A filtered code-language dataset, which is **a subset of The Stack and StackOverflow**, obtained by using a language model-based classifier (consisting of about 6B tokens)
- **A synthetic textbook dataset** consisting of <1B tokens of GPT-3.5 generated Python textbooks.
- **A small synthetic exercises dataset** consisting of ~180M tokens of Python exercises and solutions.



2. Training details and the importance of high-quality data

2.1 Filtering of existing code datasets using a transformer-based classifier

6B
a subset of The Stack and StackOverflow



1) GPT-4 is prompted to "determine its educational value for a student whose goal is to learn basic coding concepts"
2) Random Forest classifier 학습 (predicts the quality of a file/sample using its output embedding from a pretrained codegen model as features)

Educational values deemed by the filter

High educational value

```
import torch
import torch.nn.functional as F

def normalize(x, axis=-1):
    """Performs L2-Norm."""
    num = x
    denom = torch.norm(x, 2, axis, keepdim=
    True).expand_as(x) + 1e-12
    return num / denom

def euclidean_dist(x, y):
    """Computes Euclidean distance."""
    m, n = x.size(0), y.size(0)
    xx = torch.pow(x, 2).sum(1, keepdim=True)
    .expand(m, n)
    yy = torch.pow(x, 2).sum(1, keepdim=True)
    .expand(m, m).t()
    dist = xx + yy - 2 * torch.matmul(x, y.t
    ())

    dist = dist.clamp(min=1e-12).sqrt()

    return dist

def cosine_dist(x, y):
    """Computes Cosine Distance."""
    x = F.normalize(x, dim=1)
    y = F.normalize(y, dim=1)
    dist = 2 - 2 * torch.mm(x, y.t())
    return dist
```

Low educational value

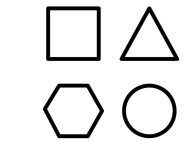
```
import re
import typing
...

class Default(object):
    def __init__(self, vim: Nvim) -> None:
        self._vim = vim
        self._denite: typing.Optional[SyncParent
    ] = None
        self._selected_candidates: typing.List[
    int] = []
        self._candidates: Candidates = []
        self._cursor = 0
        self._entire_len = 0
        self._result: typing.List[typing.Any] =
    []
        self._context: UserContext = {}
        self._bufnr = -1
        self._winid = -1
        self._winrestcmd = ''
        self._initialized = False
        self._winheight = 0
        self._winwidth = 0
        self._winminheight = -1
        self._is_multi = False
        self._is_async = False
        self._matched_pattern = ''
        self._displayed_texts: typing.List[str]
    = []
        self._statusline_sources = ''
        self._titlestring = ''
        self._ruler = False
        self._prev_action = ''
        ...
```

6B
a subset of The Stack and StackOverflow

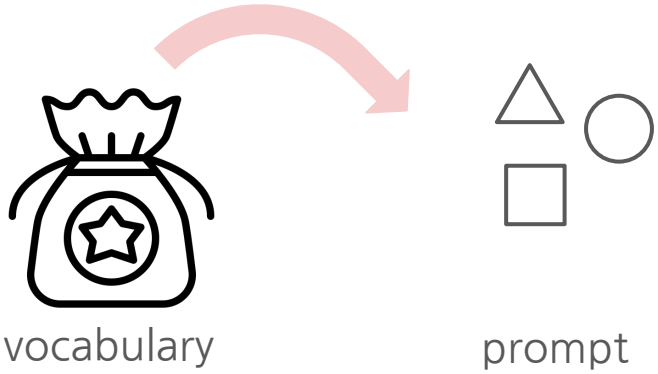
2. Training details and the importance of high-quality data

2.2 Creation of synthetic textbook-quality datasets

1B
a synthetic
textbook
dataset



high-quality dataset
- **diverse**
- **non-repetitive**



👉 **Inject randomness into the prompt to generate diverse dataset**

2. Training details and the importance of high-quality data

2.2 Creation of synthetic textbook-quality datasets

1B
a synthetic
textbook
dataset

출처: *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?*

TinyStories

- 3-4 year-old-child
- 1500 basic words
- 3 words are chosen randomly (one verb, one noun, and one adjective)

Write a short story (3-5 paragraphs) which only uses very simple words that a 3 year old child would likely understand. The story should use the verb "decorate", the noun "thunder" and the adjective "ancient". The story should have the following features: the story should contain at least one dialogue, the story has a bad ending. Remember to only use simple words!

Once upon a time, in an ancient house, there lived a girl named Lily. She loved to decorate her room with pretty things. One day, she found a big box in the attic. She opened it and saw many shiny decorations. Lily was very happy and decided to use them in her room.
As Lily was decorating her room, the sky outside became dark. There was a loud thunder sound, and Lily got scared. She ran to her mom and said, "Mommy, the thunder is so loud!" Her mom hugged her and said, "Don't worry, it will pass soon."
But the thunder did not stop. It got louder and louder, and the ancient house started to shake. Suddenly, the roof fell down on the pretty decorations. Lily was sad because her room was not pretty anymore. The end.

The synthetic textbook dataset

1B
a synthetic
textbook
dataset

- A synthetic textbook dataset consisting of **<1B tokens** of GPT-3.5 generated Python textbooks.
- Diversity is obtained by providing constraints on **topics** and **target audience** of the generated textbook.

To begin, let us define singular and nonsingular matrices. A matrix is said to be singular if its determinant is zero. On the other hand, a matrix is said to be nonsingular if its determinant is not zero. Now, let's explore these concepts through examples.

Example 1:
Consider the matrix A = np.array([[1, 2], [2, 4]]). We can check if this matrix is singular or nonsingular using the determinant function. We can define a Python function, `is_singular(A)`, which returns true if the determinant of A is zero, and false otherwise.

```
import numpy as np
def is_singular(A):
    det = np.linalg.det(A)
    if det == 0:
        return True
    else:
        return False

A = np.array([[1, 2], [2, 4]])
print(is_singular(A)) # True
```

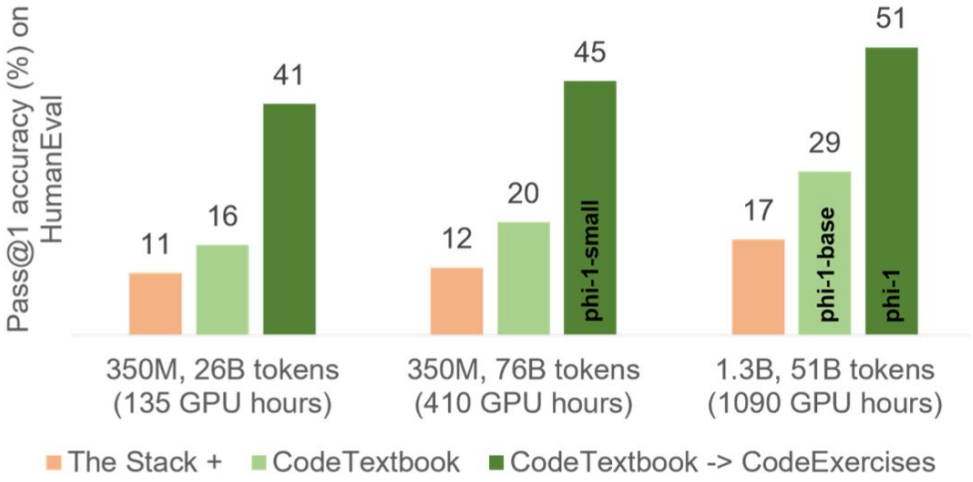
The CodeExercises dataset

180M
a small synthetic exercises dataset

- Less than **180M tokens**
- docstring of a function that needs to be completed
- generated by GPT-3.5
- to perform function completion tasks based on natural language instructions

```
def valid_guessing_letters(word: str, guesses: List[str]) -> List[str]:
    """
    Returns a list of valid guessing letters, which are letters that have not been guessed yet and
    are present in the word.
    Parameters:
    word (str): The word to guess.
    guesses (List[str]): A list of letters that have already been guessed.
    Returns:
    List[str]: A list of valid guessing letters.
    """
    valid_letters = []
    for letter in word:
        if letter not in guesses and letter not in valid_letters:
            valid_letters.append(letter)
    return valid_letters
```

2. Training details and the importance of high-quality data



- phi-1-base (1.3B)
- phi-1-small (350M)

6B
a subset of The Stack and StackOverflow

1B
a synthetic textbook dataset

- phi-1

Finetuning phi-1-base on the CodeExercises dataset

180M
a small synthetic exercises dataset

2. Training details and the importance of high-quality data

2.3 Model architecture and training

- Transformer decoder using FlashAttention
- Multi head attention(MHA) and MLP layers in parallel configuration
- rotary position embedding
- codegen-350M-mono tokenizer

Hyperparameters for phi-1 with 1.3B / 350M parameters:

- 24 / 20 layers
- Hidden dimension = 2048 / 1024
- MLP-inner dimension = 8196 / 4096
- 32 / 16 attention heads with dimension = 64 / 32
- Sequence length = 2048

2. Training details and the importance of high-quality data

2.3 Model architecture and training

Pretraining

- 1024 batch size
- maximum learning rate 1e-3
- weight decay 0.1

Finetuning

- 256 batch size
- maximum learning rate 1e-4
- weight decay 0.01

Spikes of model capability after finetuning on CodeExcercises

3. Spikes of model capability after finetuning on CodeExcercises

3.1 Finetuning improves the model's understanding

phi-1-base struggles with the logical relationships in the prompts,

while **phi-1** can interpret the question and generate the answer correctly.

Prompt:

```
""" Alice, Bob and Charles are playing games.  Alice first pick a random number from 333
to 678, Bob then pick a number starting from Alice's number to 888, Charles then pick a
number starting from 123 to Bob's number.  Alice gets one point if Alice's number minus
Charles's number is divisible by the floor of the square root of Bob's number, otherwise
Bob gets one point.  Simulate Alice's and Bob's points in 30 iterations.
"""
```

phi-1

```
points = [0, 0]
for i in range(30):
    alice_num = random.randint
    (333, 678)
    bob_num = random.randint(
    alice_num, 888)
    charles_num = random.randint
    (123, bob_num)
    if (alice_num - charles_num)
    %
        points[0] += 1
    else:
        points[1] += 1
return f"Alice: {points[0]}, Bob:
    {points[1]}"
```

phi-1-base

```
self.points = 0
self.alice = 333
self.bob = 678
self.charles = 888
self.bob_points = 0
self.alice_points = 0
self.iterations = 30
self.alice_turn = True
self.bob_turn = True
self.alice_turn_points = 0
self.bob_turn_points = 0
self.alice_turn_points_history =
    []
self.bob_turn_points_history = []
self.alice_turn_points_history
    .append(self.alice_points)
self.bob_turn_points_history
    .append(self.bob_points)
```

phi-1-small

```
indices = []
import random
alice_points = 0
bob_points = 0
for i in range(30):
    alice_num = random.randint
    (333, 678)
    bob_num = random.randint(123,
     bob_num)
    if (alice_num - bob_num) %
        alice_points += 1
    if (i %
        bob_points += 1
print(f"Alice gets {alice_points}
    points, while Bob gets {
    bob_points} points.")
```

3. Spikes of model capability after finetuning on CodeExcercises

3.2 Finetuning improves the model's ability to use external libraries

Fintuning on CodeExcercises improves the models ability to **use external libraries** such as Pygame and Tkinter.

Prompt:

```
""" Write a PyGame.  There is a ball.  At every iteration, (1).  The x-axis of the ball
increases by a random integer between (-10, 10), (2).  The y-axis of the ball increases
by a random integer between (-20, 20).  The x-axis of the ball should stay inside 0-400,
and y-axis of the ball should stay inside 0-600.  When the user press 'space', set the
x-axis of the ball to 200 and y-axis of the ball to 400.
"""
```

phi-1

```
...
screen = pygame.display.set_mode
    ((400, 600))
x, y = 0, 0
while True:
    for event in pygame.event.get
    ():
        ...
        elif event.type == pygame
    .KEYDOWN:
            if event.key ==
    pygame.K_SPACE:
                x = 200
                y = 400
    x += random.randint(-10, 10)
    y += random.randint(-20, 20)
    if x < 0:
        x = 0
    elif x > 400:
        x = 400
    if y < 0:
        y = 0
    elif y > 600:
        y = 600
    screen.fill((255, 255, 255))
    pygame.draw.circle(screen,
    (255, 0, 0), (x, y), 10)
    pygame.display.update()
```

phi-1-base

```
...
ball_x = random.randint(-10, 10)
ball_y = random.randint(-20, 20)
ball_vx = random.randint(-5, 5)
ball_vy = random.randint(-5, 5)
...
screen = pygame.display.set_mode
    ((800, 600))
...
while not done:
    for event in pygame.event.get
    ():
        ...
        elif event.type == pygame
    .KEYDOWN:
            if event.key ==
    pygame.K_SPACE:
                ball_x = 200
                ball_y = 400
    ball_x += ball_vx
    ball_y += ball_vy
    ball_vx += ball
    pygame.display.flip()
```

phi-1-small

```
...
x, y = 0, 0
dx, dy = random.randint(-10, 10),
     random.randint(-20, 20)
while True:
    screen.fill((0, 0, 0))
    pygame.draw.rect(screen,
    (255, 255, 255), (x, y, 400,
     600))
    pygame.display.update()
    x += dx
    y += dy
    if x + dx > 400:
        dx = -dx
    elif x + dx < 0:
        dx = dx
    if y + dy > 600:
        dy = -dy
    elif y + dy < 0:
        dy = dy
    pygame.time.wait(50)
```

Evaluation on unconventional problems with LLM grading

4. Evaluation on unconventional problems with LLM grading

50 new problems in the same format as HumanEval with instructions to design problems that are unlikely to appear in real-world code bases or as coding exercises.

```
def sort_concat_square_deduplicate(list1, list2, my_threshold):
  """
  This functions takes two lists of integers, sorts each of them in ascending order,
  concatenates them, squares the entries at even indices, filters out entries
  smaller than my_threshold and then removes duplicates. The resulting list is
  returned.
  """
```

(1) by using **GPT-4 as a grader**, we can leverage its knowledge and generative abilities to obtain a more fine-grained and meaningful signal of the student model's coding capabilities,

(2) it obviates the need for **tests**. Our prompt instructs the LLM to evaluate a student's solution first in a short verbal evaluation followed by grades from **0 to 10**.

4. Evaluation on unconventional problems with LLM grading

Model	Size	Training tokens	Score	HumanEval
CodeGen-Mono-350M [NPH+23]	350M	577B	0.19	12.8%
CodeGen-Mono-16.1B [NPH+23]	16.1B	577B	0.38	29.3%
Replit [Rep23]	2.7B	525B	0.37	21.9%
StarCoder [LAZ+23]	15.5B	1T	0.51	33.6%
phi-1-base	1.3B	7B	0.37	29%
phi-1-small	350M	7B	0.45	45%
phi-1	1.3B	7B	0.52	50.6%

Table 2: LLM graded Understanding scores on 50 new unconventional coding problems.

5

Data pruning for unbiased performance evaluation

5. Data pruning for unbiased performance evaluation

CodeExercises 데이터셋으로 파인튜닝한 모델이 HumanEval benchmark에서 좋은 성능을 내는 원인을 파악하고자,

CodeExercises 데이테셋 중에서 HumanEval과 비슷한 데이터셋을 제거

두가지 방법을 사용

1. N-gram overlap
2. Embedding and syntax-based similarity analysis

👉40% 가까이를 제거했음에도 우수한 성능을 보임

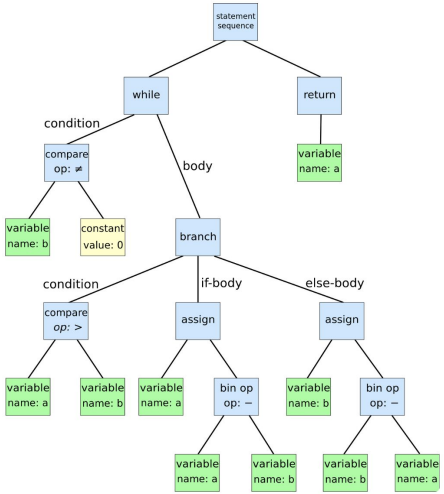
👉CodeExcercises is **not contaminated** by HumanEval

5. Data pruning for unbiased performance evaluation

5.1 N-gram overlap

- docstring을 n-gram으로 분석
- 4 overlap cases in the 13-gram ar all false positive.

HumanEval:	**CodeExercises:**
You are given a non-empty list of positive integers. Return the greatest integer that is greater than zero, and has a frequency greater than or equal to the value of the integer itself. **The frequency of an integer is the number of times it appears in the list.**	Calculates the power frequency analysis sum of a list of integers. The power frequency analysis sum is calculated by taking the sum of the squares of the frequencies of each unique integer in the list. **The frequency of an integer is the number of times it appears in the list.**



5.2 Embedding and syntax-based similarity analysis

- combination of **embedding** and **syntax-based distances**
- L2 distance between the embedding of the code snippets(pre-trained CodeGen-Mono 350M model)
- Python Docstring, function/class names, as well as the code structure
- (string) edit distance between the abstract syntax trees(ASTs) of two given code snippets

5. Data pruning for unbiased performance evaluation

5.2 Embedding and syntax-based similarity analysis

τ		Problem Count	**phi-1**	**phi-1 retrained on pruned data**	StarCoder-Prompted [LAZ+23]
0.95	similar	71	81.7%	74.6%	57.7%
	non-similar	93	26.9%	32.3%	29.0%
	total	164	50.6%	50.6%	41.5%
0.9	similar	93	63.4%	51.6%	48.4%
	non-similar	71	33.8%	36.6%	32.4%
	total	164	50.6%	45.1%	41.5%
0.85	similar	106	62.3%	52.8%	47.2%
	non-similar	58	29.3%	34.5%	31.0%
	total	164	50.6%	46.3%	41.5%
0.8	similar	116	59.5%	52.6%	45.7%
	non-similar	48	29.2%	27.1%	31.2%
	total	164	50.6%	45.1%	41.5%

Table 3: Percentage of similar versus non-similar HumanEval problems correctly solved by different models. Similarity is determined based on whether or not the corresponding HumanEval problem has any close matches inside the CodeExercises dataset (for a given τ). The problem count denotes the number of HumanEval problems within each subset. Here, τ is the threshold on AST-based match rate between codes for similarity check.

6

Conclusion

6. Conclusion

Limitation

- phi-1 is specialized in **Python coding**
- lacks the domain-specific knowledge
- less robust to variations or errors in the prompt

Challenge

- dataset covers **all the relevant content**
- dataset is truly **diverse** and **non-repetitive**
- inject **randomness** and **creativity** into the data generation process
- measure and evaluate the amount of diversity and redundancy in the data
- urgency on the ethical and social implications of training models (themselves will be used to curate data)

Q n A